PH2-C1 Colab Repair: Root Cause and Clean-Run Protocol

Repair Brief · Christ yvan Yao · Oct 20, 2026

The Sentinel run fails before any statistics are produced. This brief isolates the immediate error, traces it to two deeper methodological bugs, and sets the exact sequence for a clean Colab execution.

1. The Immediate Error

The run aborts inside the Benjamini–Hochberg step with a **TypeError**: the p-value adjustment receives a pandas `Series` where a plain numeric array is expected, so element-wise comparison against the significance level never resolves.

```
TypeError: cannot compare a Series with a scalar —
bh_adjust() received a Series, expected a 1-D numeric array
```

Corrected return contract: `bh_adjust()` and `holm_adjust()` must accept a plain numeric array (or `Series.values`) and return a NumPy array of adjusted p-values of identical length — never a scalar, never a DataFrame. Callers then index the result positionally.

2. Three-Bug Repair Map

Bug	Symptom	Repair direction
BH scalar conversion	TypeError in the multiple-testing step; adjusted values unusable downstream.	Convert input with `np.asarray()`; return an array of the same length.
Placebo type confusion	Placebo prices and PnL share one structure, so shuffled entries contaminate real returns.	Separate placebo *prices* from placebo *PnL* as distinct objects with distinct names.
Incomplete grid family	Some parameter cells are silently skipped, biasing the family of hypotheses.	Enumerate the full grid explicitly and assert the count before testing.

3. Clean-Run Protocol

1. **Restart** — Runtime → Restart runtime. Clear every stale variable and cached import.
2. **Paste once** — Paste one complete script with all fixes included; no partial cells.
3. **Run once** — Execute top to bottom without re-running earlier cells.
4. **Save** — Confirm `manifest_c1.json` is written and download it.

Do not patch cells incrementally: a mixed-state notebook reproduces the placebo contamination even after the TypeError is fixed. The manifest is the only accepted evidence of a clean run.

Exact Replacement Code Blocks

Copy-paste each block below to replace the matching function in your notebook. These edits remove the runtime error and cleanly separate placebo prices from PnL accounting.

1. Robust p-value adjusters

REPLACE BH_ADJUST() AND HOLM_ADJUST()

```
def bh_adjust(pvals):
    """Benjamini-Hochberg step-up FDR adjustment."""
    p = np.asarray(pvals, dtype=float)
    n = p.size
    order = np.argsort(p)
    ranked = p[order] * n / np.arange(1, n + 1)
    ranked = np.minimum.accumulate(ranked[::-1])[::-1]
    adj = np.empty(n)
    adj[order] = np.minimum(ranked, 1.0)
    return adj


def holm_adjust(pvals):
  """Holm step-down adjustment (family-wise error rate)."""
  p = np.asarray(pvals, dtype=float)
  n = p.size
  order = np.argsort(p)
  ranked = p[order] * (n - np.arange(n))
  ranked = np.maximum.accumulate(ranked)
  adj = np.empty(n)
  adj[order] = np.minimum(ranked, 1.0)
  return adj
```

2. Placebo price path and corrected test

REPLACE GBM_PLACEBO_PRICES_H1() AND GBM_PLACEBO_H1()

```
def gbm_placebo_prices_h1(S0, mu, sigma, n_days, n_sims, seed=0):
    """Generate placebo GBM price paths only — no trading, no PnL."""
    rng = np.random.default_rng(seed)
    dt = 1.0 / 252.0
    z = rng.standard_normal((n_sims, n_days))
    log_ret = (mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z
    return S0 * np.exp(np.cumsum(log_ret, axis=1))  # (n_sims, n_days)


def gbm_placebo_h1(prices, n_sims=2000, seed=0):
  """Run the H1 statistic on placebo paths (prices in, p-values out)."""
  return np.asarray([h1_statistic(path)[1]
                     for path in prices[:n_sims]])
```

> Key change: the placebo generator now returns **prices only**. Trade construction never runs inside the placebo path, so

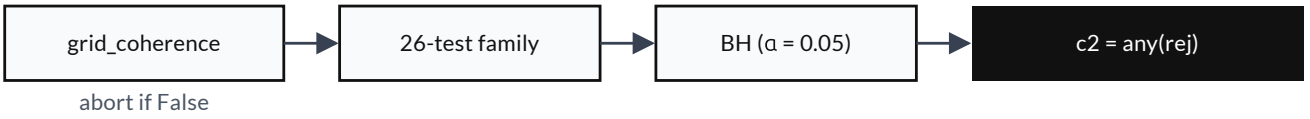
Main-Function Integration & Validation Checklist

Wire the repaired functions into **main()**, guard short trade series, apply the corrected Benjamini–Hochberg decision, then validate the run honestly.

1 · Integration edits inside main()

```
grid_ok = grid_coherence(prices, placebo_prices)   # must return True
if not grid_ok:
    raise RuntimeError("Grid coherence failed — abort before testing")


pvals  = run_hypothesis_family(pnl, placebo_pnls)  # length-26 array
rej_bh = benjamini_hochberg(pvals, alpha=0.05)
c2     = bool(np.any(rej_bh))              # corrected decision
```



2 · Safety guard for short series

```
if pnl.size < 5:
    print("SKIP: fewer than 5 trades — statistics unreliable")
    return
```

3 · Run settings, expected output & honest verdict

Setting	Smoke test	Production
Scope	3 windows, 200 iters, < 2 min	full grid, full iterations
Expected output	26 p-values print; c2 prints True/False; no traceback	same, plus final report block and clean log

PASS requires

Grid coherence True, ≥ 5 trades, and **c2 = True** after BH — at least one hypothesis survives FDR control at α = 0.05.

FAIL means

If the supplied V75 results give **c2 = False**, the strategy does **not** pass cumulative validation — report it as such; do not relax α or re-run until a pass appears.